

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

**ОТЧЕТ О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ №3
по дисциплине «Математическая логика и теория формальных
ЯЗЫКОВ»**

**Реализация контекстно-свободной грамматики
подмножества естественного языка**

Выполнил студент группы
№5130201/30101

Радионова П. В. _____

Проверил преподаватель

Востров А. В. _____

« ____ » _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Прошедшее время в корейском языке (과거 시제)	4
1.1 Частицы 은/는 и 을/를	4
1.2 Формы предложений	4
1.2.1 Утвердительное предложение	4
1.2.2 Вопросительное предложение	5
1.2.3 Отрицательное предложение	5
2 Математическое описание задачи	6
2.1 Грамматика и язык	6
2.1.1 Порождающая грамматика Хомского	6
2.2 Контекстно-свободная грамматика	8
2.3 Синтаксический анализатор	8
2.4 БНФ задаваемого множества языка	9
3 Особенности реализации	13
3.1 Структуры данных	13
3.1.1 Грамматика языка	13
3.1.2 Словарь grammar	15
3.1.3 Списки терминальных множеств	15
3.2 Класс KoreanGrammar	15
3.2.1 Конструктор класса	15
3.2.2 Метод parse_bnf	16
3.2.3 Метод _add_alternatives	17
3.2.4 Метод generate	17
3.2.5 Метод _generate_tokens	18
3.3 Класс Parser	18
3.3.1 Метод __init__	18
3.3.2 Метод parse_bnf	19
3.3.3 Вспомогательные методы разбора	20
3.3.4 Метод validate	22
3.3.5 Разбор предложения	22
3.3.6 Разбор подлежащего и дополнения	23
3.3.7 Разбор обстоятельства времени	25
3.3.8 Разбор сказуемого	26
4 Результаты работы программы	28
Заключение	30
Список использованных источников	32

Введение

Цель работы: построение контекстно-свободной грамматики подмножества естественного языка.

В данной лабораторной работе выбран корейский язык, прошедшее время (과거 시제). Разрабатываемая грамматика позволяет генерировать утвердительные, вопросительные и отрицательные предложения в соответствии с выбранным временем.

Задачи.

1. Выделить правила построения предложений в прошедшем времени корейского языка.
2. Составить грамматику выбранного языка в форме Бэкуса-Наура.
3. Реализовать программу, которая генерировала бы утвердительные, вопросительные и отрицательные предложения для заданной грамматики.
4. Реализовать программу, которая проверяла бы предложения на соответствие их заданной грамматике.

1 Прошедшее время в корейском языке (과거 시제)

Используется для выражения действия или состояния, которое произошло в прошлом.

В корейском языке прошедшее время образуется путём добавления суффиксов -았- / -었- к основе глагола или прилагательного.

Суффикс **았** применяется, когда последняя гласная в основе – это **ㅏ** или **ㅗ**. Во всех остальных случаях применяется суффикс **었**.

1.1 Частицы **은/는** и **을/를**

Частица **은/는** называется *тематической* и выделяет то, о чем пойдет речь в предложении. Тематическая частица выделяет подлежащее в предложении, указывает на субъект, выполняющий действие.

Частица добавляется в конце слова. **은** используется, когда последняя буква слова – согласная, а **는** ставится после гласных.

Пример. 저 (я) → 저는, 사람 (человек) → 사람을.

Падежные частицы **을/를** присоединяются непосредственно к объекту действия и показывают, что именно над этим предметом совершается действие.

을 используется, когда последняя буква слова – согласная, а **를** ставится после гласных.

Пример. 영화 (фильм) → 영화를, 책 (книга) → 책을.

1.2 Формы предложений

1.2.1 Утвердительное предложение

Образуется по схеме:

Subject + Adverb + Adjective + Object + [основа + **았/었 + **어요**] + ‘.’**

Subj	Adv	Adj	Obj	Основа	았/었	어요	Предл.	Пер.
저는	-	맛있는	빵을	먹	었	어요	저는 빵을 먹었어요	Я поел(-а) вкусный хлеб
그는	-	-	-	좋	았	어요	그는 좋았어요	Он был хорошим

1.2.2 Вопросительное предложение

Образуется по схеме:

Subject + Adverb + Adjective + Object + [основа + 았/었 + 어요] + '??'

Вопросительные предложения строятся аналогично утвердительным, в конце предложения ставится вопрос.

Subj	Adv	Adj	Obj	Основа	았/었	어요	Предл.	Пер.
아빠는	어제	-	영화를	보	았	어요	아빠는 어제 영화를 봤어요?	Папа вчера смотрел фильм?

1.2.3 Отрицательное предложение

Образуется по схеме:

Subject + Adverb + Adjective + Object + '안' + [основа + 았/었 + 어요] + '.'

Subj	Adv	Adj	Obj	안	Основа	았/었	어요	Предл.	Пер.
저는	어제	-	영화를	안	보	았	어요	저는 어제 영화를 안 봤어요	Я вчера не смотрел(-а) фильм

2 Математическое описание задачи

2.1 Грамматика и язык

В данной работе строится контекстно-свободная грамматика, задающая язык предложений в прошедшем времени корейского языка.

2.1.1 Порождающая грамматика Хомского

Порождающая грамматика Хомского определяется следующим образом:

$$G = \langle T, N, S, R \rangle, \text{ где}$$

- T – конечное множество терминалов (слов языка),
- N – конечное множество нетерминалов ($T \cap N = \emptyset$),
- $S \in N$ – начальный нетерминал,
- R – множество правил вывода вида $\alpha \rightarrow \beta$, где α и β – цепочки над словарем $T \cup N$.

Язык, порождаемый грамматикой G – это множество терминальных цепочек, выводимых из начального нетерминала грамматики.

Для 과거 시제 определим следующие компоненты:

$N = \{\text{sentence, punctuation, subject, subject_vowel, subject_cons, object_list, object, object_vowel, object_cons, determiner, proper_name_any, proper_name_vowel, proper_name_cons, noun_vowel, noun_cons, pronoun_vowel, pronoun_cons, adjective_list, adjective, embedded_sentence, time_opt, time, adverbial, time_point, time_phrase, specific_date, time_interval, time_unit, year, month, day, digit, number, vpast, vpast_attributive, verb_stem_ao, verb_stem_other}\},$

$T = \{\text{'안', '는', '은', '씨', '님', '를', '을', '이', '그', '저', '민수', '영희', '지수', '지민', '알렉스', '이반', '의사', '남자', '바보', '할머니', '아이', '날씨', '학생', '사람', '선생님', '여성', '책', '집', '형', '저', '너', '그', '그녀', '우리', '당신', '그들', '맛있는', '큰', '작은', '예쁜', '나쁜', '빨리', '천천히', '조용히', '에', '어제', '그때', '작년에', '아까', '예전에', '년', '월', '일', '부터', '까지', '지금까지', '동안', '시간', '주', '달', '0', '1', '2', '3', '4', '5', '6', '7', '8', '9', '10', '11', '12', '았', '었', '어요', '가', '나', '사', '자', '막', '았', '받', '보', '오', '찾', '먹', '마시', '읽', '크', '웃', '믿', '쓰', '그리', '.', '?', '!'}\}$

S – начальный нетерминал (sentence).

Множество продукций R :

sentence	→ subject time_opt adjective_list object_list vpast punctuation subject time_opt adjective_list object_list '안' vpast punctua
punctuation	→ '.' '?' '!'
subject	→ subject_vowel '는' subject_cons '은'
subject_vowel	→ pronoun_vowel determiner adjective noun_vowel determiner noun_vowel noun_vowel proper_name_vowel proper_name_any '씨'
subject_cons	→ pronoun_cons determiner adjective noun_cons determiner noun_cons noun_cons proper_name_cons proper_name_any '님'
object_list	→ objectobject_list ϵ
object	→ object_vowel '를' object_cons '을'
object_vowel	→ determiner adjective noun_vowel determiner noun_vowel noun_vowel proper_name_vowel proper_name_any '씨' determiner embedded_sentence noun_vowel embedded_sentence noun_vowel
object_cons	→ determiner adjective noun_cons determiner noun_cons noun_cons proper_name_cons proper_name_any '님' determiner embedded_sentence noun_cons embedded_sentence noun_cons
proper_name_any	→ proper_name_vowel proper_name_cons
embedded_sentence	→ subject time vpast_attributive subject vpast_attributive
time_opt	→ time ϵ
time	→ adverbial time_point time_interval
time_point	→ specific_date '에' time_phrase
specific_date	→ year '년' month '월' day '일' month '월' day '일' day '일'
time_interval	→ specific_date '부터' specific_date '까지' time_phrase '부터' '지금까지' number time_unit '동안'
year	→ digit digit digit digit
month	→ '1' '2' ... '12'
day	→ digit digit digit
number	→ digit digit digit
vpast	→ verb_stem_ao '았' '어요' verb_stem_other '였' '어요'
vpast_attributive	→ verb_stem_ao '는' verb_stem_other '은'

determiner	→ '이' '그' '저'
adjective_list	→ adjective adjective_list ϵ
adjective	→ '맛있는' '큰' '작은' '예쁜' '나쁜'
adverbial	→ '빨리' '천천히' '조용히'
time_phrase	→ '어제' '그때' '작년에' '아까' '예전에'
time_unit	→ '시간' '일' '주' '달'
digit	→ '0' '1' ... '9'
pronoun_vowel	→ '저' '너' '그' '그녀' '우리'
pronoun_cons	→ '당신' '그들'
noun_vowel	→ '의사' '남자' '바보' '할머니' '아이' '날씨'
noun_cons	→ '학생' '사람' '선생님' '여성' '책' '집' '형'
proper_name_vowel	→ '민수' '영희' '지수' '알렉스'
proper_name_cons	→ '지민' '이반'
verb_stem_ao	→ '가' '나' '사' '자' '막' '앉' '받' '보' '오' '찾'
verb_stem_other	→ '먹' '마시' '읽' '크' '웃' '믿' '쓰' '그리'

2.2 Контекстно-свободная грамматика

Грамматика является *контекстно-свободной* (тип 2 по иерархии Хомского), если для неё справедливо следующее:

- все правила имеют вид $A \rightarrow \alpha$, где $A \in N$ – одиночный нетерминал, а $\alpha \in (N \cup T)^*$.
- правые части продукций могут содержать любые комбинации нетерминалов и терминалов, но не зависят от контекста.

2.3 Синтаксический анализатор

Несмотря на то, что разработанная грамматика относится к классу КС-грамматик, она не является строго $LL(k)$ -грамматикой. В рассматриваемой грамматике существуют ϵ -продукции и правила, для которых выбор продукции нельзя сделать по фиксированному количеству входящих символов.

Для синтаксического анализа предложений в работе используется метод *рекурсивного спуска* с возвратом. На вход анализатора подается последовательность терминальных символов, из которой предварительно удаляются пробелы. Анализ начинается со стартового нетерминала и последовательно разбирает входную строку в соответствии с правилами грамматики. Каждому нетерминалу соответствует отдельная процедура разбора, которая выполняет проверку: может ли текущий фрагмент входной строки быть разобран из данного нетерминала. Если нет, то процедура откатывает позицию чтения на то место, в котором она была до попытки, и пробует следующий вариант правила.

2.4 БНФ задаваемого множества языка

Форма Бэкуса-Наура (БНФ) – форма представления контекстно-свободных формальных грамматик, в которой правые части продукций с одинаковой левой частью объединены.

Грамматика подмножества корейского языка в времени 과거 시제 в форме Бэкуса-Наура:

```
<sentence> ::= <subject> <time_opt> <adjective_list> <object_list>  
              <vpast> <punctuation> | <subject> <time_opt>  
              <adjective_list> <object_list> "안" <vpast>  
              <punctuation>
```

```
<punctuation> ::= "." | "?" | "!"
```

```
<subject> ::= <subject_vowel> "는" | <subject_cons> "은"
```

```
<subject_vowel> ::= <pronoun-vowel>  
                  | <determiner_opt> <adjective_list> <noun-vowel>  
                  | <proper_name_vowel>  
                  | <proper_name_any> "씨"
```

```
<subject_cons> ::= <pronoun-cons>  
                  | <determiner_opt> <adjective_list> <noun-cons>  
                  | <proper_name_cons>  
                  | <proper_name_any> "님"
```

```
<object_list> ::= <object> <object_list> | ε  
<object> ::= <object_vowel> "를" | <object_cons> "을"
```

```
<object_vowel> ::= <determiner_opt> <adjective_list> <noun-vowel>  
                  | <proper_name_vowel>  
                  | <proper_name_any> "씨"  
                  | <determiner_opt> <embedded_sentence> <noun-vowel>
```

```
<object_cons> ::= <determiner_opt> <adjective_list> <noun-cons>  
                  | <proper_name_cons>  
                  | <proper_name_any> "님"  
                  | <determiner_opt> <embedded_sentence> <noun-cons>
```

```
<determiner_opt> ::= <determiner> | ε  
<determiner> ::= "이" | "그" | "저"
```

```
<proper_name_any> ::= <proper_name_vowel> | <proper_name_cons>
```

```

<proper_name_vowel> ::= "민수" | "영희" | "지수" | "알렉스"
<proper_name_cons> ::= "지민" | "이반"

<noun_vowel> ::= "의사" | "남자" | "바보" | "할머니" | "아이" | "날씨"
<noun_cons> ::= "학생" | "사람" | "선생님" | "여성" | "책" | "집" | "
형"

<pronoun_vowel> ::= "저" | "너" | "그" | "그녀" | "우리"
<pronoun_cons> ::= "당신" | "그들"

<adjective_list> ::= <adjective> <adjective_list> | ε
<adjective> ::= "맛있는" | "큰" | "작은" | "예쁜" | "나쁜"

<embedded_sentence> ::= <subject> <time_opt> <adjective_list>
                        <object_list> <vpast_attributive>

<time_opt> ::= <time> | ε
<time> ::= <adverbial> | <time_point> | <time_interval>

<adverbial> ::= "빨리" | "천천히" | "조용히"

<time_point> ::= <specific_date> "에"
                | <time_phrase>

<time_phrase> ::= "어제" | "그때" | "작년에" | "아까" | "예전에"

<specific_date> ::= <year> "년" <month> "월" <day> "일"
                  | <month> "월" <day> "일"
                  | <day> "일"

<time_interval> ::= <specific_date> "부터" <specific_date> "까지"
                  | <time_phrase> "부터" "지금까지"
                  | <number> <time_unit> "동안"

<time_unit> ::= "시간" | "일" | "주" | "달"

<year> ::= <digit><digit><digit><digit>
<month> ::= "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" | "10"
            | "11" | "12"
<day> ::= <digit> | <digit><digit>

<digit> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
<number> ::= <digit> | <digit><digit>

```

<vpast> ::= <verb-stem-ao> "았" "어요" | <verb-stem-other> "었" "어
요"

<vpast_attributive> ::= <verb-stem-ao> "는" | <verb-stem-other> "은"

<verb-stem-ao> ::= "가" | "나" | "사" | "자" | "막" | "았" | "받"
| "보" | "오" | "찾"

<verb-stem-other> ::= "먹" | "마시" | "읽" | "크" | "웃" | "믿"
| "쓰" | "그리"

Примеры предложений

1. "저는 어제 큰 사람을 막았어요?"
2. "학생은 작년에 작은 선생님을 안 사았어요."
3. "저는 작년에 작은 아이를 웃었어요!"
4. "저는 2일에 맛있는 지민을 가았어요?"

Пример дерева разбора приведен на рис. 1.

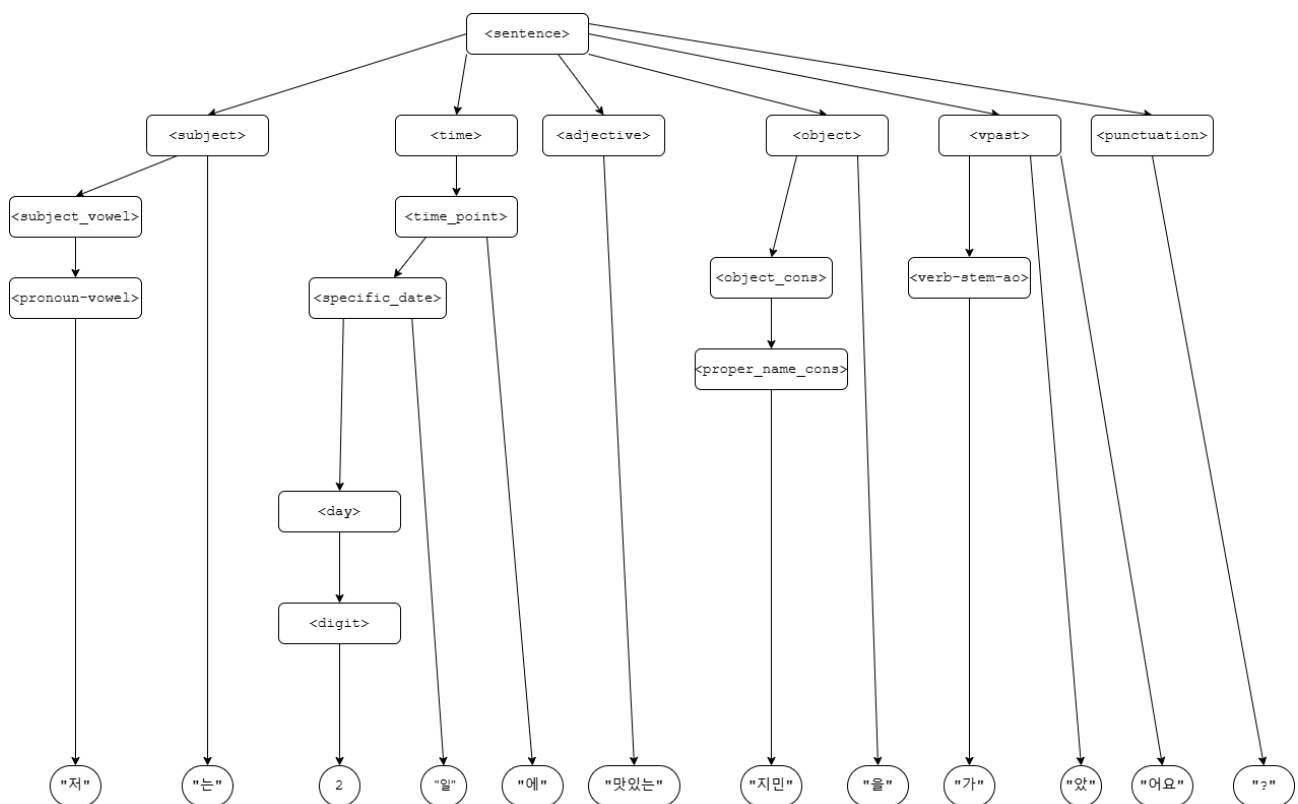


Рис. 1: Дерево разбора

В описанной грамматике присутствует рекурсия в следующих правилах вывода:

<object> ::= <object_vowel> "를" | <object_cons> "을"

<object_vowel> ::= <determiner_opt> <adjective_list> <noun-vowel>
| <proper_name_vowel>
| <proper_name_any> "씨"
| <determiner_opt> <embedded_sentence> <noun-vowel>

<object_cons> ::= <determiner_opt> <adjective_list> <noun-cons>
| <proper_name_cons>
| <proper_name_any> "님"
| <determiner_opt> <embedded_sentence> <noun-cons>

<embedded_sentence> ::= <subject> <time_opt> <adjective_list>
<object_list> <vpast_attributive>

3 Особенности реализации

3.1 Структуры данных

В программе используются следующие основные структуры данных.

3.1.1 Грамматика языка

GRAMMAR_BNF — строка, хранящая грамматику языка. Является источником терминальных множеств в программе: функция `parse_bnf` разбирает её при старте и заполняет все словари парсера.

Строка GRAMMAR_BNF (фрагмент)

```
1 GRAMMAR_BNF = ""
2 <sentence> ::= <subject> <time_opt> <adjective_list> <object_list>
3             <vpast> <punctuation> | <subject> <time_opt>
4             <adjective_list> <object_list> "안" <vpast>
5             <punctuation>
6
7 <punctuation> ::= "." | "?" | "!"
8
9 <subject> ::= <subject_vowel> "는" | <subject_cons> "은"
10
11 <subject_vowel> ::= <pronoun-vowel>
12                  | <determiner_opt> <adjective_list> <noun-vowel>
13                  | <proper_name_vowel>
14                  | <proper_name_any> "씨"
15
16 <subject_cons> ::= <pronoun-cons>
17                  | <determiner_opt> <adjective_list> <noun-cons>
18                  | <proper_name_cons>
19                  | <proper_name_any> "님"
20
21 <object_list> ::= <object> <object_list> | ε
22 <object> ::= <object_vowel> "를" | <object_cons> "을"
23
24 <object_vowel> ::= <determiner_opt> <adjective_list> <noun-vowel>
25                  | <proper_name_vowel>
26                  | <proper_name_any> "씨"
27                  | <determiner>? <embedded_sentence> <noun-vowel>
28
29 <object_cons> ::= <determiner_opt> <adjective_list> <noun-cons>
30                  | <proper_name_cons>
31                  | <proper_name_any> "님"
32                  | <determiner_opt> <embedded_sentence> <noun-cons>
33
34 <determiner_opt> ::= <determiner> | ε
35 <determiner> ::= "이" | "그" | "저"
36
37 <proper_name_any> ::= <proper_name_vowel> | <proper_name_cons>
38 <proper_name_vowel> ::= "민수" | "영희" | "지수" | "알렉스"
39 <proper_name_cons> ::= "지민" | "이반"
```

```

40
41 <noun-vowel> ::= "의사" | "남자" | "바보" | "할머니" | "아이" | "날씨"
42 <noun-cons> ::= "학생" | "사람" | "선생님" | "여성" | "책" | "집" | "형"
43
44 <pronoun-vowel> ::= "저" | "너" | "그" | "그녀" | "우리"
45 <pronoun-cons> ::= "당신" | "그들"
46
47 <adjective_list> ::= <adjective> <adjective_list> | ε
48 <adjective> ::= "맛있는" | "큰" | "작은" | "예쁜" | "나쁜"
49
50 <embedded_sentence> ::= <subject> <time_opt> <adjective_list>
51                             <object_list> <vpast_attributive>
52
53 <time_opt> ::= <time> | ε
54 <time> ::= <adverbial> | <time_point> | <time_interval>
55
56 <adverbial> ::= "빨리" | "천천히" | "조용히"
57
58 <time_point> ::= <specific_date> "에"
59                 | <time_phrase>
60
61 <time_phrase> ::= "어제" | "그때" | "작년에" | "아까" | "예전에"
62
63 <specific_date> ::= <year> "년" <month> "월" <day> "일"
64                 | <month> "월" <day> "일"
65                 | <day> "일"
66
67 <time_interval> ::= <specific_date> "부터" <specific_date> "까지"
68                 | <time_phrase> "부터" "지금까지"
69                 | <number> <time_unit> "동안"
70
71 <time_unit> ::= "시간" | "일" | "주" | "달"
72
73 <year> ::= <digit><digit><digit><digit>
74 <month> ::= "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" | "10"
75           | "11" | "12"
76 <day> ::= <digit> | <digit><digit>
77
78 <digit> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
79 <number> ::= <digit> | <digit><digit>
80
81 <vpast> ::= <verb-stem-ao> "았" "어요" | <verb-stem-other> "었" "어요"
82
83 <vpast_attributive> ::= <verb-stem-ao> "는" | <verb-stem-other> "은"
84
85 <verb-stem-ao> ::= "가" | "나" | "사" | "자" | "막" | "았" | "받"
86                 | "보" | "오" | "찾"
87 <verb-stem-other> ::= "먹" | "마시" | "읽" | "크" | "웃" | "믿"
88                   | "쓰" | "그리"
89 ""

```

3.1.2 Словарь grammar

Основной структурой данных генератора является словарь `grammar`, формируемый методом `parse_bnf`.

Ключами словаря являются нетерминалы грамматики, а значениями — списки продукций данного нетерминала.

Пример заполненного словаря `grammar`

```
{
  "<sentence>": [
    "<subject>", "<vpast>", "<punctuation>",
    "<subject>", "안", "<vpast>", "<punctuation>"
  ],
  "<punctuation>": [
    ".",
    "?",
    "!"
  ]
}
```

3.1.3 Списки терминальных множеств

В классе `Parser` терминалы грамматики сохраняются в отдельных списках:

Списки терминалов

```
self.punctuation
self.determiner
self.adjective
self.pronoun_vowel
self.pronoun_cons
self.noun_vowel
self.noun_cons
...
```

Каждый список содержит допустимые терминалы соответствующей категории грамматики.

3.2 Класс `KoreanGrammar`

Генерация предложений на корейском языке в прошедшем времени реализована в классе `KoreanGrammar`.

3.2.1 Конструктор класса

- **Вход:** экземпляр класса, грамматика языка.
- **Выход:** объект класса `KoreanGrammar` с заполненным словарём грамматики.

Конструктор выполняет начальную инициализацию объекта. После создания экземпляра класса строка с грамматикой автоматически разбирается методом `parse_bnf`, а результат сохраняется в поле `self.grammar`. Полученная структура используется генератором предложений.

```
1      Конструктор класса
2  def __init__(self, grammar_text):
    self.grammar = self.parse_bnf(grammar_text)
```

3.2.2 Метод `parse_bnf`

- **Вход:** экземпляр класса, текст грамматики языка.
- **Выход:** словарь правил грамматики.

Метод выполняет разбор BNF-грамматики. Строка с описанием правил построчно анализируется, после чего формируется словарь, где каждому нетерминалу соответствует список возможных продукций. Метод поддерживает как объявление новых правил, так и продолжение альтернатив через символ `"|"`.

```
1      Метод parse_bnf
2  def parse_bnf(self, text):
3      grammar = {}
4
5      current_rule = None
6
7      for raw_line in text.splitlines():
8          line = raw_line.strip()
9
10         if not line:
11             continue
12
13         # новое правило
14         if "::=" in line:
15             left, right = line.split("::=", 1)
16
17             current_rule = left.strip()
18             grammar[current_rule] = []
19
20             self._add_alternatives(grammar[current_rule], right)
21
22         # продолжение предыдущего правила
23         elif line.startswith("|"):
24             if current_rule is None:
25                 raise ValueError(f"Некорректная BNF строка: {line}")
26
27             self._add_alternatives(grammar[current_rule], line[1:])
28
29     return grammar
```


3.2.3 Метод `_add_alternatives`

- **Вход:** экземпляр класса, список правил, текст продукции.
- **Выход:** модифицированный список правил.

Метод выделяет альтернативные продукции грамматики и разбивает их на отдельные токены. С помощью регулярного выражения определяются терминалы и нетерминалы, после чего каждая продукция добавляется в список правил текущего нетерминала.

```
1         Метод _add_alternatives
2     def _add_alternatives(self, rule_list, text):
3
4         alternatives = [alt.strip() for alt in text.split("|")]
5
6         for alt in alternatives:
7             if not alt:
8                 continue
9
10            tokens = []
11
12            # <nonterminal> или "terminal"
13            for token in re.findall(r'<[^>]+>|"[^"]+"', alt):
14
15                # terminal
16                if token.startswith('"'):
17                    tokens.append(token[1:-1])
18
19                # nonterminal
20                else:
21                    tokens.append(token)
22
23            rule_list.append(tokens)
```

3.2.4 Метод `generate`

- **Вход:** экземпляр класса, начальный нетерминал.
- **Выход:** строка — сгенерированное предложение.

Метод запускает процесс генерации предложения. Для указанного стартового символа рекурсивно строится цепочка терминалов с помощью метода `_generate_tokens`. После генерации выполняется объединение токенов в корректную строку корейского предложения.

```
1         Метод generate
2     def generate(self, start_symbol="<sentence>"):
3         tokens = self._generate_tokens(start_symbol)
4
5         result = self.smart_join(tokens)
6
7         return result.strip()
```

3.2.5 Метод `_generate_tokens`

- **Вход:** экземпляр класса, текущий символ грамматики, текущая глубина рекурсии.
- **Выход:** список терминальных токенов.

Метод рекурсивно раскрывает нетерминалы грамматики. Для каждого нетерминала случайным образом выбирается одна из возможных продукций, после чего выполняется дальнейшая генерация её элементов. Если символ является терминалом, он сразу добавляется в результирующий список. Для предотвращения бесконечной рекурсии используется ограничение глубины вложенности.

```
1      def _generate_tokens(self, symbol, depth=0):
2          if depth > 15:
3              return []
4
5          if symbol not in self.grammar:
6              return [symbol]
7
8          production = random.choice(self.grammar[symbol])
9
10         result = []
11
12         for token in production:
13             result.extend(self._generate_tokens(token, depth + 1))
14
15         return result
```

3.3 Класс `Parser`

Для проверки сгенерированных или введенных пользователем предложений на соответствие грамматике был реализован класс `Parser`.

3.3.1 Метод `__init__`

- **Вход:** экземпляр класса, грамматика языка.
- **Выход:** заполненные словари терминалов.

Конструктор класса выполняет начальную инициализацию синтаксического анализатора.

После создания объекта текст грамматики передаётся методу `parse_bnf`, который извлекает из БНФ все терминальные множества языка.

Дополнительно формируется объединённый список `_all_proper_names`, содержащий все имена собственные, а поле `remaining` используется для хранения непрочитанной части входной строки во время анализа.

```

1  def __init__(self, grammar_text):
2      t = self.parse_bnf(grammar_text)
3      self.punctuation = t["punctuation"]
4      self.determiner = t["determiner"]
5      self.proper_name_vowel = t["proper_name_vowel"]
6      self.proper_name_cons = t["proper_name_cons"]
7      self.noun_vowel = t["noun_vowel"]
8      self.noun_cons = t["noun_cons"]
9      self.pronoun_vowel = t["pronoun_vowel"]
10     self.pronoun_cons = t["pronoun_cons"]
11     self.adjective = t["adjective"]
12     self.adverbial = t["adverbial"]
13     self.time_phrase = t["time_phrase"]
14     self.time_unit = t["time_unit"]
15     self.verb_stem_ao = t["verb_stem_ao"]
16     self.verb_stem_other = t["verb_stem_other"]
17     self._all_proper_names = self.proper_name_vowel + self.proper_name_cons
18     self.remaining = ""

```

3.3.2 Метод parse_bnf

- **Вход:** экземпляр класса, грамматика языка.
- **Выход:** словарь терминальных множеств.

Метод выполняет частичный разбор БНФ-грамматики и извлекает из неё терминальные символы.

Если продукция содержит только терминальный символ и не содержит вложенных нетерминалов, этот терминал добавляется в соответствующий список. Таким образом формируется словарь, в котором ключом является имя нетерминала, а значением — список допустимых терминалов данной категории.

```

1  def parse_bnf(self, bnf_text):
2      terminals = {}
3      blocks = re.split(r'\n(?:=<)', bnf_text.strip())
4      for block in blocks:
5          block = block.strip()
6          if not block:
7              continue
8          m = re.match(r'<([\w_]+)>\s*::=\s*(.*)', block, re.DOTALL)
9          if not m:
10             continue
11          name = m.group(1)
12          rhs = m.group(2)
13          alternatives = [a.strip() for a in re.split(r'\s*\|\\s*', rhs)]
14          token_list = []
15          for alt in alternatives:
16              if re.search(r'<[\w_]+>', alt):

```

```

17         continue
18         tokens = re.findall(r'"([^\"]+)"', alt)
19         if len(tokens) == 1:
20             token_list.append(tokens[0])
21     if token_list:
22         terminals[name] = token_list
23     return terminals

```

3.3.3 Вспомогательные методы разбора

Метод `match`

- **Вход:** строка или список строк `target` — токен или набор токенов, которые нужно попробовать прочитать.
- **Выход:** `True`, если токен найден и прочитан; `False`, если нет.

Метод проверяет, начинается ли непрочитанный остаток строки (`remaining`) с одного из заданных токенов. Если да — этот токен удаляется из начала `remaining` и метод возвращает `True`. Если нет — `remaining` остаётся без изменений и метод возвращает `False`.

Когда на вход передаётся список, токены перебираются от самого длинного к самому короткому. Это необходимо, чтобы избежать ошибочного чтения короткого префикса вместо более длинного слова. Например, если в списке есть и `마시` и `마`, то при обычном переборе `마` может быть прочитана первой, и строка `마시였어요` окажется разобрана неверно. Перебор от длинных к коротким исключает такую ситуацию.

Метод `match`

```

1 def match(self, target):
2     if isinstance(target, list):
3         for option in sorted(target, key=len, reverse=True):
4             if self.remaining.startswith(option):
5                 self.remaining = self.remaining[len(option):]
6                 return True
7         return False
8     else:
9         if self.remaining.startswith(target):
10             self.remaining = self.remaining[len(target):]
11             return True
12         return False

```

Методы `_try` и `_seq`

- **Вход:** список вариантов разбора (для `_try`) или список частей правила (для `_seq`).
- **Выход:** `True` при успехе. При неудаче строка возвращается в исходное состояние и метод возвращает `False`.

Метод `_try` соответствует правилам БНФ вида $A / B / C$: пробует варианты по очереди, пока какой-нибудь не подойдёт. Метод `_seq` соответствует правилам вида $A B C$: проверяет все части подряд, и если хотя бы одна не подошла — возвращает строку как было до вызова.

Методы `_try` и `_seq`

```

1 def _try(self, *alternatives):
2     snap = self._save()
3     for alt in alternatives:
4         if alt():
5             return True
6         self._restore(snap)
7     return False
8
9 def _seq(self, *parts):
10    snap = self._save()
11    for p in parts:
12        ok = p() if callable(p) else self.match(p)
13        if not ok:
14            self._restore(snap)
15            return False
16    return True

```

Метод `_seq_bt`

- **Вход:** список частей правила, среди которых некоторые могут быть необязательными.
- **Выход:** `True`, если удалось разобрать строку хотя бы при одном сочетании необязательных частей; иначе `False`.

Расширенный вариант `_seq` для правил с необязательными частями. Метод по очереди перебирает все возможные комбинации. Это нужно, когда необязательная часть, прочитанная сразу же, мешает дальнейшему разбору. Например, в строке `이/받` символ `이/` можно прочитать как определитель, но тогда останется `받`, что не подходит. Метод `_seq_bt` в этом случае пробует другой вариант: не читать `이/` как определитель, и тогда правило успешно прочтёт имя `이/받` целиком.

Метод `_seq_bt`

```

1 def _seq_bt(self, *parts):
2     snap_outer = self._save()
3     opt_idx = [i for i, pt in enumerate(parts)
4                 if isinstance(pt, Parser._Opt)]
5     n_opt = len(opt_idx)
6     for mask_value in range(2 ** n_opt - 1, -1, -1):
7         self._restore(snap_outer)
8         taken = {opt_idx[k]: bool((mask_value >> k) & 1)
9                  for k in range(n_opt)}
10        ok_all = True
11        for i, pt in enumerate(parts):

```

```

12         if isinstance(pt, Parser._Opt):
13             if taken[i] and not pt.fn():
14                 ok_all = False; break
15         else:
16             ok = pt() if callable(pt) else self.match(pt)
17             if not ok:
18                 ok_all = False; break
19     if ok_all:
20         return True
21     self._restore(snap_outer)
22     return False

```

3.3.4 Метод validate

- **Вход:** экземпляр класса, строка `sentence`.
- **Выход:** `True`, если предложение соответствует грамматике, иначе `False`.

Удаляет пробелы из входной строки, запускает разбор от стартового правила и проверяет, что вся строка прочитана до конца.

Метод validate

```

1 def validate(self, sentence):
2     if not sentence or not sentence.strip():
3         return False
4     self.remaining = sentence.replace(" ", "")
5     try:
6         ok = self.parse_sentence()
7         return bool(ok) and self.remaining == ""
8     except Exception:
9         return False

```

3.3.5 Разбор предложения

Метод parse_sentence

- **Вход:** экземпляр класса.
- **Выход:** `True`, если строка целиком соответствует структуре «подлежащее + время + прилагательные + дополнения + **안** + сказуемое + знак препинания».

Сначала пробуются разбор с временем и с отрицанием, затем без них. Прилагательные и дополнения сначала читаются как можно больше, а если дальше разбор не сложился — метод откатывается к меньшему количеству и пробует ещё раз.

Метод parse_sentence

```

1 def parse_sentence(self):
2     snap_outer = self._save()
3     for take_time in (True, False):

```

```

4         for take_neg in (True, False):
5             self._restore(snap_outer)
6             if not self.parse_subject(): return False
7             if take_time and not self.parse_time(): continue
8             adj_positions = [self._save()]
9             while self.match(self.v.adjective):
10                 adj_positions.append(self._save())
11             for adj_snap in reversed(adj_positions):
12                 self._restore(adj_snap)
13                 obj_positions = [self._save()]
14                 while True:
15                     snap_o = self._save()
16                     if self.parse_object(depth=0):
17                         obj_positions.append(self._save())
18                     else:
19                         self._restore(snap_o); break
20                 for obj_snap in reversed(obj_positions):
21                     self._restore(obj_snap)
22                     if take_neg and not self.match("안"): continue
23                     if not self.parse_vpast(): continue
24                     if not self.match(self.v.punctuation): continue
25                     if self.remaining == "": return True
26         self._restore(snap_outer)
27         return False

```

3.3.6 Разбор подлежащего и дополнения

Метод parse_subject

- **Вход:** экземпляр класса.
- **Выход:** True, если в начале строки распознано подлежащее с частицей $\frac{\text{는}}{\text{은}}$.

Соответствует методам `gen_subject_vowel` и `gen_subject_cons` генератора. Все возможные виды подлежащего перечислены вместе с подходящей частицей в одном списке из 8 вариантов.

Метод parse_subject

```

1 def parse_subject(self):
2     v = self.v
3     all_names = self._all_proper_names
4     return self._try(
5         lambda: self._seq(lambda: self.match(all_names), "씨", "는"),
6         lambda: self._seq(lambda: self.match(v.pronoun_vowel), "는"),
7         lambda: self._seq(lambda: self.match(v.proper_name_vowel), "는"),
8         lambda: self._seq(self._parse_np_vowel, "는"),
9         lambda: self._seq(lambda: self.match(all_names), "님", "은"),
10        lambda: self._seq(lambda: self.match(v.pronoun_cons), "은"),
11        lambda: self._seq(lambda: self.match(v.proper_name_cons), "은"),
12        lambda: self._seq(self._parse_np_cons, "은"),
13    )

```

Метод `parse_object`

- **Вход:** экземпляр класса, `depth` (текущая глубина рекурсии).
- **Выход:** `True`, если распознано дополнение с частицей `를/을`.

Устроен так же, как `parse_subject`: перечисляет все 8 видов дополнения вместе с падежной частицей. Среди них есть и вариант с вложенным предложением (`embedded_sentence`).

```
1 def parse_object(self, depth):
2     v = self.v
3     all_names = self._all_proper_names
4     return self._try(
5         lambda: self._seq(lambda: self.match(all_names), "씨", "를"),
6         lambda: self._seq(lambda: self._parse_object_embedded_vowel(depth),
7             "를"),
8         lambda: self._seq(self._parse_np_vowel, "를"),
9         lambda: self._seq(lambda: self.match(v.proper_name_vowel), "를"),
10        lambda: self._seq(lambda: self.match(all_names), "님", "을"),
11        lambda: self._seq(lambda: self._parse_object_embedded_cons(depth),
12            "을"),
13        lambda: self._seq(self._parse_np_cons, "을"),
14        lambda: self._seq(lambda: self.match(v.proper_name_cons), "을"),
15    )
```

Методы `_parse_np_vowel` и `_parse_np_cons`

- **Вход:** экземпляр класса.
- **Выход:** `True`, если распознана именная группа *определитель прилагательное существительное*.

Определитель и прилагательное необязательны, поэтому они помечены как `_opt`, а сама последовательность собирается через `_seq_bt`, который перебирает все четыре сочетания.

```
1 def _parse_np_vowel(self):
2     return self._seq_bt(
3         self._opt(lambda: self.match(self.v.determiner)),
4         self._opt(lambda: self.match(self.v.adjective)),
5         lambda: self.match(self.v.noun_vowel),
6     )
7
8 def _parse_np_cons(self):
9     return self._seq_bt(
10        self._opt(lambda: self.match(self.v.determiner)),
11        self._opt(lambda: self.match(self.v.adjective)),
12        lambda: self.match(self.v.noun_cons),
13    )
```


Метод `parse_embedded_sentence`

- **Вход:** экземпляр класса, `depth` (текущая глубина рекурсии).
- **Выход:** `True`, если распознан причастный оборот *подлежащее + время + глагол в атрибутивной форме*.

Соответствует методу `gen_embedded_sentence`. Время необязательно, поэтому используется `_seq_bt`.

```
1  def parse_embedded_sentence(self, depth):
2      return self._seq_bt(
3          self.parse_subject,
4          self._opt(self.parse_time),
5          self.parse_vpast_attributive,
6      )
```

3.3.7 Разбор обстоятельства времени

Метод `parse_time`

- **Вход:** экземпляр класса.
- **Выход:** `True`, если распознан временной маркер.

Сначала пробует временной интервал, потом точку времени, потом наречие. Длинные варианты пробуются раньше коротких, чтобы парсер не остановился на короткой части, которую правильнее прочесть как длинную конструкцию (например, не прочесть *어제* как наречие, если на самом деле дальше идёт *어제부터 지금까지*).

```
1  def parse_time(self):
2      return self._try(
3          self.parse_time_interval,
4          self.parse_time_point,
5          lambda: self.match(self.v.adverbial),
6      )
```

Методы `parse_time_point` и `parse_specific_date`

- **Вход:** экземпляр класса.
- **Выход:** `True`, если распознан конкретный момент времени.

`parse_time_point` пробует прочесть дату с маркером *오* или готовое наречие времени. `parse_specific_date` распознаёт три формы даты, перечисляя их от самой полной к самой короткой.

```

1  _____ Методы parse_time_point и parse_specific_date _____
2  def parse_time_point(self):
3      return self._try(
4          lambda: self._seq(self.parse_specific_date, "에"),
5          lambda: self.match(self.v.time_phrase),
6      )
7
8  def parse_specific_date(self):
9      return self._try(
10         lambda: self._seq(self._digits, "년", self._digits,
11             "월", self._digits, "일"),
12         lambda: self._seq(self._digits, "월", self._digits, "일"),
13         lambda: self._seq(self._digits, "일"),
14     )

```

Метод parse_time_interval

- **Вход:** экземпляр класса.
- **Выход:** True, если распознан временной интервал.

Соответствует методу gen_time_interval. Распознаёт три формы временных интервалов с частицами 부터, 까지, 동안.

```

1  _____ Метод parse_time_interval _____
2  def parse_time_interval(self):
3      return self._try(
4          lambda: self._seq(self.parse_specific_date, "부터",
5              self.parse_specific_date, "까지"),
6          lambda: self._seq(lambda: self.match(self.v.time_phrase),
7              "부터지금까지"),
8          lambda: self._seq(self._digits,
9              lambda: self.match(self.v.time_unit), "동안"),
10     )

```

3.3.8 Разбор сказуемого

Методы parse_vpast и parse_vpast_attributive

- **Вход:** экземпляр класса.
- **Выход:** True, если распознана соответствующая глагольная форма.

Соответствуют методам gen_vpast и gen_vpast_attributive. Парсер читает основу глагола и проверяет, что после неё идёт нужный суффикс: 았어요 или 었어요 для прошедшего времени, 는 или 은 для атрибутивной формы.

```

1  _____ Методы parse_vpast и parse_vpast_attributive _____
2  def parse_vpast(self):
3      return self._try(
4          lambda: self._seq(lambda: self.match(self.v.verb_stem_ao), "았어요"),
5          lambda: self._seq(lambda: self.match(self.v.verb_stem_other), "었어요"),
6      )

```

```
5     )
6
7 def parse_vpast_attributive(self):
8     return self._try(
9         lambda: self._seq(lambda: self.match(self.v.verb_stem_ao), "ㄴ"),
10        lambda: self._seq(lambda: self.match(self.v.verb_stem_other), "은"),
11    )
```

4 Результаты работы программы

На рис. 2 приведен пример генерации предложений и их автоматическая проверка.

```
=====
ГЕНЕРАЦИЯ И ПРОВЕРКА
=====
1. 그들은 맛있는 날씨는 3835년 10월 24일에 그리은 남자를 보았어요?
   Проверка: ВЕРНО

2. 책은 영희씨를 믿었어요.
   Проверка: ВЕРНО

3. 민수님은 작은 이반을 그리었어요!
   Проверка: ВЕРНО

4. 예쁜 선생님은 알렉스님을 자았어요.
   Проверка: ВЕРНО

5. 이반씨는 예쁜 작은 믿었어요.
   Проверка: ВЕРНО

6. 알렉스씨는 영희를 안 마시었어요.
   Проверка: ВЕРНО

7. 저는 9월 6일부터 6월 24일까지 그 그는 아까 사는 책을 크었어요!
   Проверка: ВЕРНО

8. 큰 집은 조용히 이반님을 안 았았어요?
   Проверка: ВЕРНО

9. 그녀는 보았어요?
   Проверка: ВЕРНО
```

Рис. 2: Проверка сгенерированных предложений

На рис. 3 приведен пример проверки введенных пользователем предложений и обработки некорректных предложений.

```

=====
Введите предложение для проверки.
Для выхода из программы введите 'q'.

>> Ввод: 저는 9월 6일부터 6월 24일까지 그 그는 아까 사는 책을 크었어요!
      Результат: ВЕРНО - Предложение соответствует грамматике.

>> Ввод: 저는 9월 6일부터 6월 24일까지 그 그는 아까 사는 책을 크었어요
      Результат: ОШИБКА - Предложение содержит синтаксическую ошибку.

>> Ввод: 저는 9월 일부터 월 24일까지 그 그는 아까 사는 책을 크었어요!
      Результат: ОШИБКА - Предложение содержит синтаксическую ошибку.

>> Ввод: 9월 6일부터 6월 24일까지 그 그는 아까 사는 책을 크었어요!
      Результат: ОШИБКА - Предложение содержит синтаксическую ошибку.

>> Ввод: 저는 9월 6일부터 6월 24일까지 그 그는 아까 사는 책을 크었어요...
      Результат: ОШИБКА - Предложение содержит синтаксическую ошибку.

>> Ввод: 저는 9월 6일부터 6월 24일까지 그 그는 아까 사는 책을 크었어요.
      Результат: ВЕРНО - Предложение соответствует грамматике.

```

Рис. 3: Проверка введенных пользователем предложений

На рис. 4 приведен пример обработки пустого предложения. При вводе символа 'q' программа завершает свою работу.

```

>> Ввод:
      Результат: ОШИБКА - пустая строка, предложение не соответствует грамматике.

>> Ввод: q

Process finished with exit code 0

```

Рис. 4: Проверка ввода пустой строки

Заключение

В ходе выполнения лабораторной работы была построена контекстно-свободная грамматика, описывающая подмножество корейского языка в прошедшем времени (과거 시제), и реализована программа, которая проверяет предложения на соответствие грамматике и порождает случайные предложения, соответствующие правилам грамматики.

В рамках работы также были выделены правила построения предложений корейского языка в прошедшем времени: структура подлежащего и дополнения с тематическими и падежными частицами и образование прошедшего времени глагола. Составленная грамматика была описана в нотации Бэкуса-Наура.

Построенная грамматика является контекстно-свободной (тип 2 по иерархии Хомского). Грамматика не принадлежит классу $LL(k)$ ни при каком фиксированном k : в ней присутствуют ϵ -продукции и правила с совпадающими префиксами альтернатив, из-за которых однозначный выбор продукции по конечному числу символов вперёд невозможен.

Для синтаксического анализа применён метод рекурсивного спуска с возвратом: каждому нетерминалу грамматики соответствует отдельная процедура разбора, а при неудаче любого варианта позиция в строке откатывается и пробуются следующая продукция. Анализатор работает непосредственно с символьной строкой без её предварительного разбиения на токены.

Достоинства программы

1. Программа корректно обрабатывает правила корейской морфонологии, такие как выбор тематических частиц (은/는) и падежных маркеров (을/를) в зависимости от наличия падцима – согласной буквы внизу корейского слога.
2. Система способна генерировать сложные предложения с вложенными конструкциями.

Недостатки программы

1. Терминальные символы жестко зафиксированы внутри кода, что делает невозможным добавление новых слов пользователем без изменения исходного текста программы.
2. Реализованная грамматика описывает только очень ограниченный набор конструкций прошедшего времени.

Масштабирование программы

1. Добавление новых временных форм (настоящее, будущее), вежливых стилей речи и более сложных союзов для связи предложений.
2. Добавление возможности загрузки предложений на проверку из файла.

Список литературы

- [1] Секция «Телематика» ВШТИИ. – [\[Электронный ресурс\]](#) (дата обращения: 23.03.2026).
- [2] **Карпов Ю. Г.** Теория и технология программирования. Основы построения трансляторов. – СПб.: БХВ-Петербург, 2005 – 272 с.
- [3] **Ahn Jean-myung, Lee Kyung-ah, Han Hoo-young** Korean Grammar in Use. Beginning to Early Intermediate. – DARAKWON – 345 с.